

MagistralaCAN4

Wysokowydajny sniffer CAN / CAN FD

Uczenie asocjacyjne • Silnik Lua • Akceleracja GPU • WebSocket

Wersja 2.0.0 | C++17 | Qt 6.10 | Linux socketCAN

CustomLabs

May 7, 2026

Contents

1	Schemat blokowy	3
2	Wprowadzenie	3
3	Architektura systemu	3
3.1	Przepływ ramki CAN	3
3.2	Komponenty rdzenia	4
4	Zakładki aplikacji	4
4.1	Ruch CAN	4
4.2	Szczegóły ramki	4
4.3	Analiza offline	5
4.4	Dashboard CAN	5
5	Uczenie asocjacyjne — szczegółowy opis	5
5.1	Filozofia działania	5
5.2	Wektor cech	5
5.3	Miara podobieństwa	5
5.4	Tabele analityczne	6
5.5	Klastrowanie	6
5.6	Eksport i import	6
6	Silnik skryptowy Lua	6
6.1	API	7
6.2	Pełny przykład	7
6.3	Generowanie skryptów z uczenia asocjacyjnego	7
7	Serwer WebSocket	8
7.1	Format JSON	8
7.2	Klient testowy (JavaScript)	8
8	Kompilacja i uruchomienie	8
8.1	Wymagania	8
8.2	Budowanie	8
8.3	Konfiguracja vcan0	9
9	Struktura katalogów	9
10	Skróty klawiszowe	9
11	Technologie	10

1 Schemat blokowy

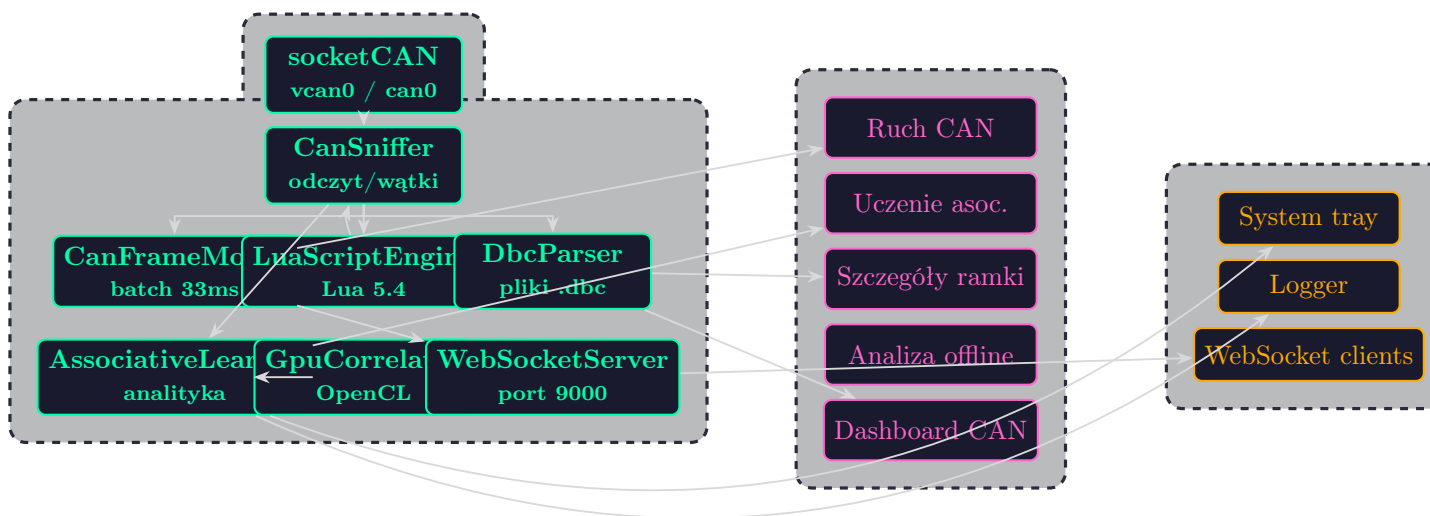


Figure 1: Schemat blokowy architektury MagistralaCAN4

2 Wprowadzenie

MagistralaCAN4 to zaawansowane narzędzie do analizy magistrali CAN, zaprojektowane z myślą o inżynierach motoryzacyjnych, badaczach systemów wbudowanych i entuzjastach reverse engineeringu protokołów komunikacyjnych. Aplikacja łączy przechwytywanie ruchu CAN w czasie rzeczywistym z zaawansowaną analizą danych, uczeniem asocjacyjnym i zdalnym dostępem przez WebSocket.

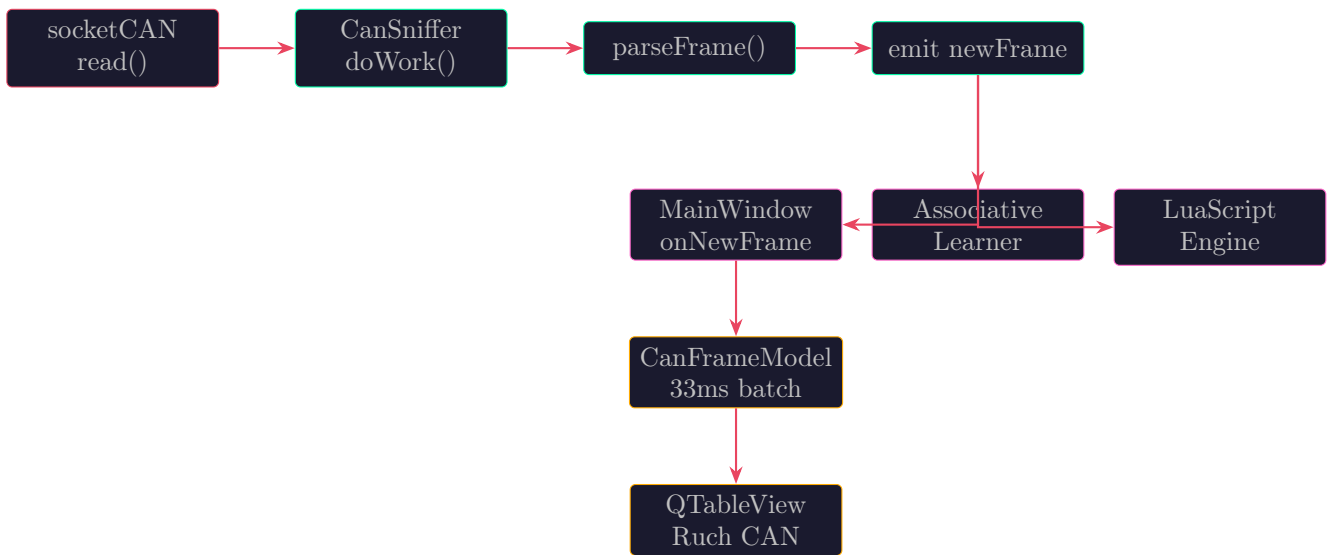
Kluczowe cechy

- Wielowątkowy odczyt CAN/CAN FD przez socketCAN
- Zaawansowane uczenie asocjacyjne (14 tabel analitycznych, klastrowanie, PCA, anomalie)
- Silnik skryptowy Lua z API CAN (sendFrame, log, getTick)
- Akceleracja GPU przez OpenCL (fallback CPU)
- Serwer WebSocket streamujący ramki jako JSON
- Parser DBC z integracją sygnałów w dashboardzie
- Analiza offline plików candump
- Interaktywny edytor ramek z siatką bitów

3 Architektura systemu

3.1 Przepływ ramki CAN

Poniższy diagram sekwencji przedstawia drogę ramki od socketCAN przez wszystkie warstwy systemu:



3.2 Komponenty rdzenia

Komponent	Odpowiedzialność
CanSniffer	Wielowątkowy odczyt/zapis socketCAN (QtConcurrent::run). Emituje sygnał <code>newFrame</code> dla każdej ramki.
CanFrameModel	QAbstractTableModel z 6 kolumnami. Nadpisywanie wg ID, aktualizacja wsadowa co 33 ms.
AssociativeLearner	Uczenie asocjacyjne, korelacje, klastrowanie, anomalie, predykcja.
LuaScriptEngine	Interpreter Lua 5.4 z API CAN: <code>sendFrame</code> , <code>log</code> , <code>getTick</code> .
DbcParser	Parsowanie plików <code>.dbc</code> , dekompozycja sygnałów.
GpuCorrelator	Obliczenia macierzy podobieństw na GPU (OpenCL) lub CPU.
WebSocketServer	Serwer WebSocket na porcie 9000, stream JSON ramek CAN.
Logger	Zapis zdarzeń do <code>~/magistrala_can4.log</code> .

4 Zakładki aplikacji

4.1 Ruch CAN

Główna tabela pokazująca ruch na magistrali w czasie rzeczywistym. Zawiera 6 kolumn: ID, Extended, RTR, DLC, Data (szesnastkowo), Timestamp. Checkbox **Nadpisywanie** w toolbarze kontroluje, czy nowsze ramki o tym samym ID zastępują starsze. Przycisk **Eksportuj candump** zapisuje bieżący stan tabeli do pliku w standardowym formacie candump.

4.2 Szczegóły ramki

Widok pojedynczej ramki CAN (po kliknięciu w tabeli). Zawiera:

- **Siatkę bitów 8×8** — każdy bajt jest wizualizowany jako 8 bitów (0/1)
- **Podświetlanie zmian** — bity zmienione względem poprzedniej ramki tego samego ID
- **Sygnały DBC** — po wczytaniu pliku `.dbc` pokazuje nazwy i wartości sygnałów
- **Edycję ramek** — kliknięcie bajtu umożliwia edycję (hex lub przełączanie bitów)
- **Wysyłanie** — przycisk Wyślij zmodyfikowaną ramkę

4.3 Analiza offline

Odtwarzacz plików candump:

- Wczytywanie standardowych plików candump
- Suwak prędkości: od $0.1\times$ do $10\times$ prędkości oryginalnej
- Tryb oryginalnych timestampów lub stałego interwału
- Tryb krokowy: przycisk **Następna ramka** / klawisz **Enter**
- Integracja z AssociativeLearner i LuaScriptEngine

4.4 Dashboard CAN

Dashboard sygnałów DBC aktualizowany na żywo. Po wczytaniu pliku .dbc automatycznie tworzy panele (QLabel) dla wszystkich sygnałów, wyświetlając nazwę, aktualną wartość i jednostkę.

5 Uczenie asocjacyjne — szczegółowy opis

5.1 Filozofia działania

System uczy się powiązań między **zmiennymi użytkownika** (np. temperatura, prędkość obrotowa) a **ramkami CAN**. Użytkownik rejestruje zdarzenia (wartość zmiennej + okno czasowe ramek CAN), a system znajduje identyfikatory CAN, których charakterystyka koreluje ze zmianami zmiennej. Dodatkowo można rejestrować **braki zdarzeń** jako tło kontrastowe, które obniża score kandydatów podobnych do tła.

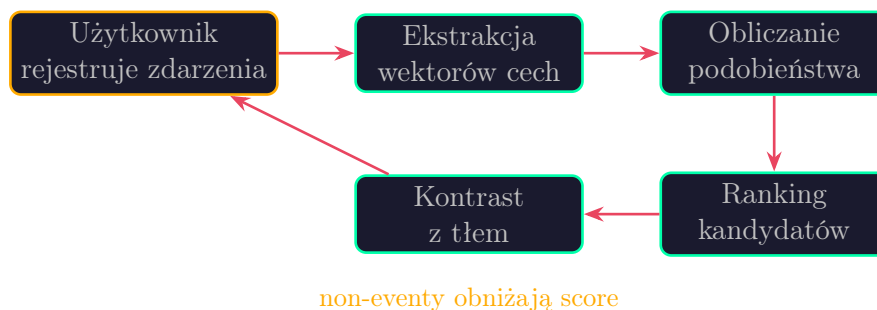


Figure 2: Cykl uczenia asocjacyjnego

5.2 Wektor cech

Dla każdego ID w oknie zdarzenia budowany jest 67-elementowy wektor cech:

3 cechy statystyczne

64 cechy sygnałowe

f_0 : liczba ramek	f_1 : śr. odstęp	f_2 : odch. stand.	$f_{3...66}$: śr. bajtów 64 bajty / 255
----------------------	--------------------	----------------------	--

5.3 Miara podobieństwa

Podobieństwo między dwoma wektorami cech v_1, v_2 obliczane jest jako podobieństwo cosinusowe:

$$\text{sim}(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\|_2 \cdot \|v_2\|_2 + \varepsilon}$$

gdzie $\varepsilon = 10^{-6}$ zapobiega dzieleniu przez zero.

5.4 Tabele analityczne



- **Korelacja wartość-bajt** — Pearson r między zmienną a bajtami
- **Sekwencje** — najczęstsze sekwencje ID (bigramy/trigramy)
- **Korelacja międzybajtowa** — Pearson r między parami bajtów
- **Mutual Information** — zależności nieliniowe (histogram 10×10, zrównoleglony)
- **MIC** — Maximal Information Coefficient (siatki 2×2 do 8×8)
- **Predykcja wartości** — regresja $y = a \cdot \text{byte} + b$, tylko $|r| > 0.8$
- **Markov** — najbardziej prawdopodobny następny ID
- **Klastrowanie** — k-średnich, auto-K (łokieć), PCA+2D
- **Anomalie** — model normalny, Z-score, próg konfigurowalny

5.5 Klastrowanie

Moduł klastrowania grupuje okna czasowe w 3 klastry na podstawie 5-wymiarowych wektorów cech: liczby ramek, liczby unikalnych ID, entropii rozkładu ID oraz czasu trwania okna. Dodatkowo dostępne są:

- **Auto K (łokieć)** — automatyczny dobór optymalnej liczby klastrów metodą łokcia (WCSS)
- **PCA + k-średnich** — redukcja do 2D metodą potęgową, wizualizacja scatter plot

5.6 Eksport i import



6 Silnik skryptowy Lua

Silnik Lua 5.4 jest zintegrowany przez LuaScriptEngine. Skrypty są ładowane w toolbarze (Wczytaj skrypt Lua). Każda ramka CAN wyzwała funkcję `onFrame(id, data, timestamp)`.

6.1 API

Wysyła ramkę CAN na magistralę.

Parametry: id (integer), data_table (tabela bajtów, indeksowana od 1, maks. 64).

Zwraca: true lub false, error_msg.

```
1 local ok, err = sendFrame(0x123, {0xAA, 0xBB, 0xCC})
2 if not ok then log("Bład: " .. err) end
```

Zapisuje wiadomość do logu (~magistrala_can4.log).

```
1 log("Odebrano ramke ID: " .. string.format("0x%X", id))
```

Zwraca liczbę milisekund od uruchomienia silnika Lua.

```
1 local uptime_ms = getTick()
```

6.2 Pełny przykład

```
1 function onFrame(id, data, timestamp)
2     if id == 0x123 then
3         if data[1] > 0x80 then
4             log("Wysoka wartosc: " .. data[1])
5             sendFrame(0x456, {0x01, 0x02})
6         end
7     end
8 end
```

Listing 1: Monitorowanie ID 0x123 z automatyczną odpowiedzią

6.3 Generowanie skryptów z uczenia asocjacyjnego

Przycisk Generuj skrypt Lua automatycznie tworzy skrypt z regułami dla top-5 kandydatów:

```
1 -- Automatycznie wygenerowane reguly asocjacyjne
2 -- Zmienna: temperatura
3 function onFrame(id, data, timestamp)
4     if id == 0x18F then
5         log(string.format("Zdarzenie: ID 0x18F, bajt 3 = %d", data[4]))
6     end
7 end
```

Listing 2: Automatycznie wygenerowany skrypt

7 Serwer WebSocket

Serwer `WebSocketServer` nasłuchuje domyślnie na porcie **9000** i streamuje każdą ramkę CAN jako kompaktowy JSON do wszystkich podłączonych klientów.

7.1 Format JSON

Listing 3: JSON emitowany dla każdej ramki

```
{
  "type": "frame",
  "id": 4660,
  "extended": false,
  "rtr": false,
  "error": false,
  "fd": false,
  "dlc": 8,
  "data": "a1b2c3d4e5f60708",
  "dataBytes": [161,178,195,212,229,246,7,8],
  "timestamp": 1715123456789
}
```

7.2 Klient testowy (JavaScript)

```
1 const ws = new WebSocket('ws://localhost:9000');
2 ws.onmessage = (e) => {
3   const frame = JSON.parse(e.data);
4   console.log('ID: 0x${frame.id.toString(16)}, DLC: ${frame.dlc}');
5 };
```

Listing 4: Klient WebSocket w przeglądarce

8 Kompilacja i uruchomienie

8.1 Wymagania

GCC 9+ / Clang 10+

CMake $\geq$ 3.16

Qt 6.2+
Widgets, Charts
Concurrent, WebSockets

Lua 5.3/5.4

OpenCL 1.2+

XCB

Linux
socketCAN

moduły can/vcan

8.2 Budowanie

Listing 5: Komendy budowania

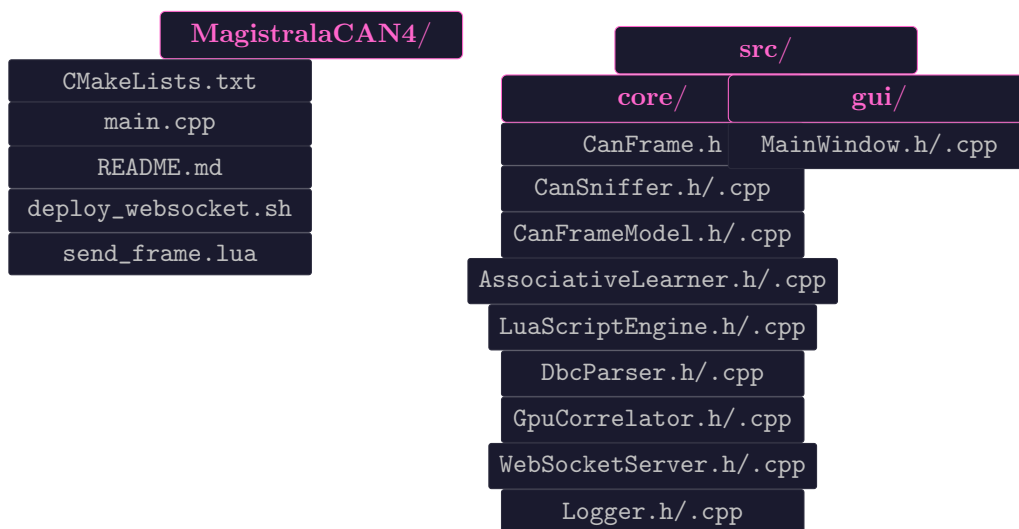
```
git clone <repo-url>
cd MagistralaCAN4
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
./MagistralaCAN4
```


8.3 Konfiguracja vcan0

Listing 6: Tworzenie wirtualnego interfejsu CAN

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
# generowanie ramek testowych:
sudo apt install can-utils
cangen vcan0 -v -I 123 -L 8 -g 10
```

9 Struktura katalogów



10 Skróty klawiszowe

Skrót	Działanie	Kontekst
Ctrl+Shift+E	Zarejestruj zdarzenie	Globalny
Ctrl+Shift+D	Zarejestruj brak zdarzenia	Globalny
Enter	Następna ramka (OfflineAnalyzer)	Zakładka offline

11 Technologie

